

# Platform Engineering for LLM Applications

Observability, security, and cost control are all critical for LLM applications. Here are a few things you could do to enhance their efficiency.



**A**fter the article on Ollama and Docker carried in the July 2026 issue of OSFY, a few readers wrote in with the same question. A gentleman from Coimbatore put it best: “Sir, model has been running for two months and everybody is happy, but now my CFO is asking me questions I cannot answer.”

I got a call in April that said the same thing, but from the finance side. A friend who runs engineering at a logistics company in Pune, someone I have known since our Wipro days, rang on a Thursday to tell me their AI spend had gone from Rs 42,000 in February to Rs 1.1 million by mid-April. He wanted to know why and how that happened.

I asked which service was responsible. Nobody knew. One of his leads said they could grep the application logs and try to work it

out, which is a reasonable instinct but would not have helped. Three teams had shipped LLM features in six weeks. None had instrumented anything beyond what Django gave them. In an access log, a request that makes fourteen model calls looks the same as one that makes a single call.

It was the agent. Someone had built one that could call five internal APIs and keep looping until it liked its own answer, with no ceiling on the retry. We found the flaw in two days once traces existed. Before that, we were guessing while the meter ran.

I have had that conversation about fifteen times now. This article is about what I usually end up explaining.

## Most of your platform is fine

Some consultants will tell you AI changes everything and your stack needs

rethinking. Usually they have a proposal ready. Kubernetes is still Kubernetes. ArgoCD does not care what runs in the pod. Prometheus scrapes as it always did. Most of what you built stays.

But three things break.

**They fail without noise:** A retail client of ours had a support assistant that spent eleven days describing a returns policy withdrawn in 2024. Dashboards were green throughout, uptime was fine, and the error rate was zero. Standard monitoring read a well-formed 200 as success. But here it only meant the service had replied.

**The cost signal gets delayed by a month:** On a normal service, wasteful code shows up as load within minutes. Someone sends 40,000 tokens of context in every request because that is easier than writing a retrieval filter, and nothing happens. No spike, no alert.